

Lab 4 - Connect your device to the Internet

Connect your IoT device to MQTT

The first part of adding Internet control to your nightlight is connecting it to an MQTT broker.

Task

Connect your device to an MQTT broker.

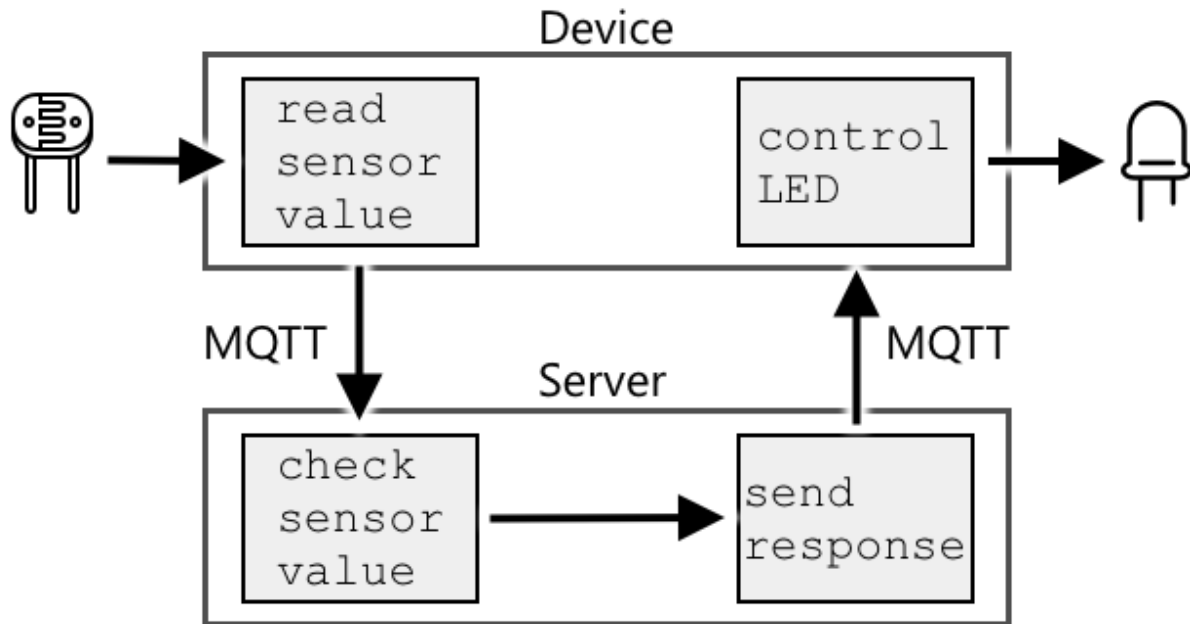
In this part of the lesson, you will connect your IoT nightlight to the internet to allow it to be remotely controlled. Later in this lesson, your IoT device will send a telemetry message over MQTT to a public MQTT broker with the light level, where it will be picked up by some server code that you will write. This code will check the light level and send a command message back to the device telling it to turn the LED on or off.

The real-world use case for such a setup could be to gather data from multiple light sensors before deciding to turn on lights, in a location that has a lot of lights, such as a stadium. This could stop the lights from being turned on if only one sensor was covered by clouds or a bird, but the other sensors detected enough light.

✅ What other situations would require data from multiple sensors to be evaluated before sending commands?

Rather than dealing with the complexities of setting up an MQTT broker as part of this assignment, you can use a public test server that runs [Eclipse Mosquitto](https://eclipse.org/mosquitto/), an open-source MQTT broker. This test broker is publicly available at test.mosquitto.org, and doesn't require an account to be set up, making it a great tool for testing MQTT clients and servers.

🧑 This test broker is public and not secure. Anyone could be listening to what you publish, so it should not be used with any data that needs to be kept private



Control your nightlight over the Internet - Virtual IoT Hardware

The IoT device needs to be coded to communicate with *test.mosquitto.org* using MQTT to send telemetry values with the light sensor reading, and receive commands to control the LED.

In this part of the lesson, you will connect your Raspberry Pi or virtual IoT device to an MQTT broker.

Install the MQTT client package

To communicate with the MQTT broker, you need to install an MQTT library pip package either on your Pi, or in your virtual environment if you are using a virtual device.

Task

Install the pip package

1. Open the nightlight project in VS Code.
2. Ensure the terminal is running the virtual environment.
3. Run the following command to install the MQTT pip package:

```
pip3 install paho-mqtt
```

Code the device

The device is ready to code.

Task

Write the device code.

1. Add the following import to the top of the app.py file:

```
import paho.mqtt.client as mqtt
```

The paho.mqtt.client library allows your app to communicate over MQTT.

2. Add the following code after the definitions of the light sensor and LED:

```
id = '<ID>'
client_name = id + 'nightlight_client'
```

Replace <ID> with a unique ID that will be used the name of this device client, and later for the topics that this device publishes and subscribes to. The *test.mosquitto.org* broker is public and used by many people, including other students working through this assignment. Having a unique MQTT client name and topic names ensures your code won't clash with anyone elses. You will also need this ID when you are creating the server code later in this assignment.



You can use a website like [GUIDGen](https://guidgen.io/) to generate a unique ID.

The client_name is a unique name for this MQTT client on the broker.

3. Add the following code below this new code to create an MQTT client object and connect to the MQTT broker:

```
mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
print("MQTT connected!")
```

This code creates the client object, connects to the public MQTT broker, and starts a processing loop that runs in a background thread listening for messages on any subscribed topics.

4. Save and run the code in the same way as you ran the code from the previous lab. Make sure the CounterFit app is running and the light sensor and LED have been created on the correct pins.
5. (.venv) → nightlight python app.py

MQTT connected!

Light level: 0

Light level: 0

👤 Your code should look like this:

```
#Created by Florin ANTON ← Your name here
import time
from counterfit_connection import CounterFitConnection
from counterfit_shims_grove.grove_light_sensor_v1_2 import
GroveLightSensor
from counterfit_shims_grove.grove_led import GroveLed
import paho.mqtt.client as mqtt

CounterFitConnection.init('127.0.0.1', 5000)

light_sensor = GroveLightSensor(0)
led = GroveLed(5)
id = '0a1215d5-5aa1-4b46-b347-870418f0835a'
client_name = id + 'nightlight_client'

mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
print("MQTT connected!")

while True:
    light = light_sensor.light
    print('Light level:', light)

    if light < 300:
        led.on()
    else:
        led.off()

    time.sleep(1)
```

😊 You have successfully connected your device to an MQTT broker. Take a screenshot of VSCode with the app running and save it as 1.jpg

Send telemetry from your IoT device

The next part in adding Internet control to your nightlight is sending the light level telemetry to the MQTT broker on a telemetry topic.

Task - send telemetry from your IoT device

Send light level telemetry to the MQTT broker.

Data is sent encoded as JSON - short for JavaScript Object Notation, a standard for encoding data in text using key/value pairs.

✅ If you've not come across JSON before, you can learn more about it on the [JSON.org documentation](https://www.json.org/).

Control your nightlight over the Internet - Virtual IoT Hardware

In this part of the lesson, you will send telemetry with light levels from your Raspberry Pi or virtual IoT device to an MQTT broker.

Publish telemetry

The next step is to create a JSON document with telemetry and send it to the MQTT broker.

Task

Publish telemetry to the MQTT broker.

1. Open the nightlight project in VS Code.
2. Ensure the terminal is running the virtual environment.
3. Add the following import to the top of the app.py file:

```
import json
```

The json library is used to encode the telemetry as a JSON document.

4. Add the following after the client_name declaration:

```
client_telemetry_topic = id + '/telemetry'
```

The client_telemetry_topic is the MQTT topic the device will publish light levels to.

5. Replace the contents of the while True: loop at the end of the file with the following:

```
while True:
    light = light_sensor.light
    telemetry = json.dumps({'light' : light})
    print("Sending telemetry ", telemetry)
    mqtt_client.publish(client_telemetry_topic, telemetry)
    time.sleep(5)
```

This code packages the light level into a JSON document and publishes it to the MQTT broker. It then sleeps to reduce the frequency that messages are sent.

6. Save and run the code in the same way as you ran the code from the previous part of the assignment. Make sure the CounterFit app is running and the light sensor and LED have been created on the correct pins.
7. (.venv) → nightlight python app.py

MQTT connected!

Sending telemetry {"light": 0}

Sending telemetry {"light": 0}

 Your code should look like this:

```
#Created by Florin ANTON ← Your name here
import time
import json
import paho.mqtt.client as mqtt
from counterfit_connection import CounterFitConnection
from counterfit_shims_grove.grove_light_sensor_v1_2 import GroveLightSensor
from counterfit_shims_grove.grove_led import GroveLed

CounterFitConnection.init('127.0.0.1', 5000)

light_sensor = GroveLightSensor(0)
led = GroveLed(5)
id = '0a1215d5-5aa1-4b46-b347-870418f0835a'
client_name = id + 'nightlight_client'
client_telemetry_topic = id + '/telemetry'

mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
print("MQTT connected!")

while True:
    light = light_sensor.light
    telemetry = json.dumps({'light' : light})
    print("Sending telemetry ", telemetry)
    mqtt_client.publish(client_telemetry_topic, telemetry)
    time.sleep(5)
```

😊 You have successfully sent telemetry from your device. Take a screenshot of VSCode with the app running and save it as 2.jpg

Receive telemetry from the MQTT broker

There's no point in sending telemetry if there's nothing on the other end to listen for it. The light level telemetry needs something listening to it to process the data. This 'server' code is the kind of code you will deploy to a cloud service as part of a larger IoT application, but here you are going to run this code locally on your computer (or on your Pi if you are coding directly on there). The server code consists of a Python app that listens to telemetry messages over MQTT with light levels. Later in this lesson you will make it reply with a command message with instructions to turn the LED on or off.

✅ Do some research: What happens to MQTT messages if there is no listener?

Task - configure a Python virtual environment

Configure a Python virtual environment and install the MQTT pip packages.

1. From your terminal or command line, run the following in your home directory:

```
mkdir nightlight-server
```

```
cd nightlight-server
```

2. Now run the following to create a virtual environment in the .venv folder

```
python3 -m venv .venv
```

👤 You need to explicitly call python3 to create the virtual environment just in case you have Python 2 installed in addition to Python 3 (the latest version). If you have Python2 installed then calling python will use Python 2 instead of Python 3

3. Activate the virtual environment:

```
source ./venv/bin/activate
```

👤 These commands should be run from the same location you ran the command to create the virtual environment. You will never need to navigate into the .venv folder, you should always run the activate command and any commands to install packages or run code from the folder you were in when you created the virtual environment.

4. Once the virtual environment has been activated, the default python command will run the version of Python that was used to create the virtual environment. Run the following to get the version:

```
python --version
```

The output will be similar to the following:

```
(.venv) → nightlight-server$ python --version
```

```
Python 3.12.3
```

🧑 Your Python version may be different - as long as it's version 3.6 or higher you are good. If not, delete this folder, install a newer version of Python and try again.

5. Run the following commands to install the pip package for [Paho-MQTT](#), a popular MQTT library.

```
pip install paho-mqtt
```

This pip package will only be installed in the virtual environment, and will not be available outside of this.

Write the server code

The server code can now be written in Python.

Task - write the server code

Write the server code.

1. From your terminal or command line, run the following inside the virtual environment to create a Python file called app.py:

```
touch app.py
```

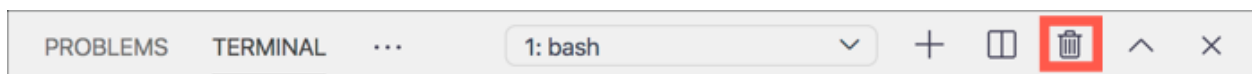
2. Open the current folder in VS Code:

```
code .
```

3. When VS Code launches, it will activate the Python virtual environment. This will be reported in the bottom status bar:

Python 3.9.1 64-bit ('.venv': venv)

4. If the VS Code Terminal is already running when VS Code starts up, it won't have the virtual environment activated in it. The easiest thing to do is kill the terminal using the **Kill the active terminal instance** button:



5. Launch a new VS Code Terminal by selecting *Terminal -> New Terminal, or pressing CTRL+`. The new terminal will load the virtual environment, with the call to activate this appearing in the terminal. The name of the virtual environment (.venv) will also be in the prompt:

→ nightlight-server\$ source .venv/bin/activate

(.venv) → nightlight-server\$

6. Open the app.py file from the VS Code explorer and add the following code:

```
import json
import time

import paho.mqtt.client as mqtt

id = '<ID>'

client_telemetry_topic = id + '/telemetry'
client_name = id + '_nightlight_server'

mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_name)
mqtt_client.connect('test.mosquitto.org')

mqtt_client.loop_start()

def handle_telemetry(client, userdata, message):
    payload = json.loads(message.payload.decode())
    print("Message received:", payload)

mqtt_client.subscribe(client_telemetry_topic)
mqtt_client.on_message = handle_telemetry

while True:
    time.sleep(2)
```

Replace <ID> on line 6 with the unique ID you used when creating your device code.

 This **must** be the same ID that you used on your device, or the server code won't subscribe or publish to the right topic.

This code creates an MQTT client with a unique name, and connects to the *test.mosquitto.org* broker. It then starts a processing loop that runs in on a background thread listening for messages on any subscribed topics.

The client then subscribes to messages on the telemetry topic, and defines a function that is called when a message is received. When a telemetry message is received, the `handle_telemetry` function is called, printing the message received to the console.

Finally an infinite loop keeps the application running. The MQTT client is listening to messages on a background thread and runs all the time the main application is running.

7. Save the app. From the VS Code terminal, run the following to run your Python app:

```
python app.py
```

The app will start listening to messages from the IoT device.

8. Make sure your device is running and sending telemetry messages. Adjust the light levels detected by your physical or virtual device. Messages being received will be printed to the terminal.

```
(.venv) → nightlight-server python app.py
```

```
Message received: {'light': 0}
```

```
Message received: {'light': 400}
```

The `app.py` file in the `nightlight` virtual environment has to be running for the `app.py` file in the `nightlight-server` virtual environment to receive the messages being sent.

Send commands to the MQTT broker

The next step for our Internet controlled nightlight is for the server code to send a command back to the IoT device to control the light based on the light levels it senses.

1. Open the server code in VS Code
2. Add the following line after the declaration of the `client_telemetry_topic` to define which topic to send commands to:

```
server_command_topic = id + '/commands'
```

3. Add the following code to the end of the `handle_telemetry` function:

```
command = { 'led_on' : payload['light'] < 300 }  
print("Sending message:", command)  
  
client.publish(server_command_topic, json.dumps(command))
```

This sends a JSON message to the command topic with the value of `led_on` set to true or false depending on if the light is less than 300 or not. If the light is less than 300, true is sent to instruct the device to turn the LED on.

4. Run the code as before
5. Adjust the light levels detected by your physical or virtual device. Messages being received and commands being sent will be written to the terminal:

```
(.venv) → nightlight-server python app.py
```

```
Message received: {'light': 0}
```

```
Sending message: {'led_on': True}
```

```
Message received: {'light': 400}
```

```
Sending message: {'led_on': False}
```

 The telemetry and commands are being sent on a single topic each. This means telemetry from multiple devices will appear on the same telemetry topic, and commands to multiple devices will appear on the same commands topic. If you wanted to send a command to a specific device, you could use multiple topics, named with a unique device id, such as /commands/device1, /commands/device2. That way a device can listen on messages just meant for that one device.

Handle commands on the IoT device

Now that commands are being sent from the server, you can now add code to the IoT device to handle them and control the LED.

Control your nightlight over the Internet - Virtual IoT Hardware

In this part of the lesson, you will subscribe to commands sent from an MQTT broker to your virtual IoT device.

Subscribe to commands

The next step is to subscribe to the commands sent from the MQTT broker and respond to them.

Task

Subscribe to commands.

1. Open the nightlight project in VS Code.
2. Ensure the terminal is running the virtual environment.
3. Add the following code after the definitions of the client_telemetry_topic:

```
server_command_topic = id + '/commands'
```

The `server_command_topic` is the MQTT topic the device will subscribe to receive LED commands.

4. Add the following code just above the main loop, after the `mqtt_client.loop_start()` line:

```
def handle_command(client, userdata, message):
    payload = json.loads(message.payload.decode())
    print("Message received:", payload)
    if payload['led_on']:
        led.on()
    else:
        led.off()

mqtt_client.subscribe(server_command_topic)
mqtt_client.on_message = handle_command
```

This code defines a function, `handle_command`, that reads a message as a JSON document and looks for the value of the `led_on` property. If it is set to `True` the LED is turned on, otherwise it is turned off.

The MQTT client subscribes on the topic that the server will send messages on and sets the `handle_command` function to be called when a message is received.

 The `on_message` handler is called for all topics subscribed to. If you later write code that listens to multiple topics, you can get the topic that the message was sent to from the message object passed to the handler function.

5. Save and run the code in the same way as you ran the code from the previous part of the assignment. Make sure the CounterFit app is running and the light sensor and LED have been created on the correct pins.
6. Adjust the light levels detected by your physical or virtual device. Messages being received and commands being sent will be written to the terminal. The LED will also be turned on and off depending on the light level.

 Your code should look like this:

```
#Created by Florin ANTON ← Your name here
import time
import json
import paho.mqtt.client as mqtt
from counterfit_connection import CounterFitConnection
from counterfit_shims_grove.grove_light_sensor_v1_2 import
GroveLightSensor
from counterfit_shims_grove.grove_led import GroveLed
```

```

CounterFitConnection.init('127.0.0.1', 5000)

light_sensor = GroveLightSensor(0)
led = GroveLed(5)
id = '0a1215d5-5aa1-4b46-b347-870418f0835a'
client_name = id + 'nightlight_client'
client_telemetry_topic = id + '/telemetry'
server_command_topic = id + '/commands'

mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_name)
mqtt_client.connect('test.mosquitto.org')
mqtt_client.loop_start()
print("MQTT connected!")

def handle_command(client, userdata, message):
    payload = json.loads(message.payload.decode())
    print("Message received:", payload)
    if payload['led_on']:
        led.on()
    else:
        led.off()

mqtt_client.subscribe(server_command_topic)
mqtt_client.on_message = handle_command

while True:
    light = light_sensor.light
    telemetry = json.dumps({'light' : light})
    print("Sending telemetry ", telemetry)
    mqtt_client.publish(client_telemetry_topic, telemetry)
    time.sleep(5)

```

😊 You have successfully coded your device to respond to commands from an MQTT broker. Having the client and the server running take a screenshot of VSCode with the **server** app running and save it as 3.jpg, and take a screenshot of VSCode with the **client** app running and save it as 4.jpg

Upload all 4 screenshots to the assignment on moodle.